同济大学计算机系

操作系统课程实验报告

学　　号 2353761

姓　　名 王皙晶

专　　业 计算机科学与技术

授课老师 方钰

实验八：UNIX V6++文件系统

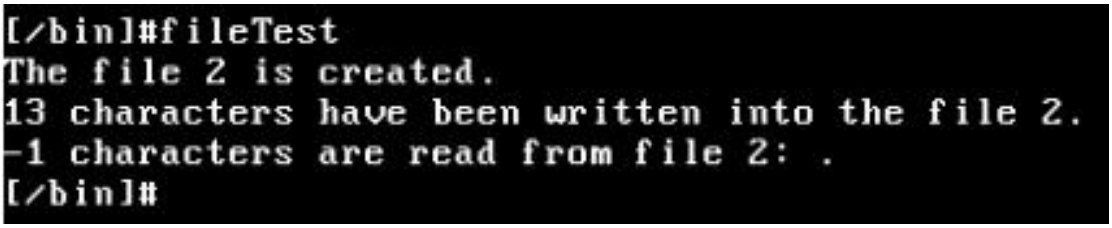
一、实验目的

通过在 UNIX V6++实验环境中编写使用文件管理相关的系统调用或库函数的应用程序，进一步了解 UNIX 文件管理的工作过程。

二、实验结果

（1）完成实验 4.1，并回答其中的 1 个问题。

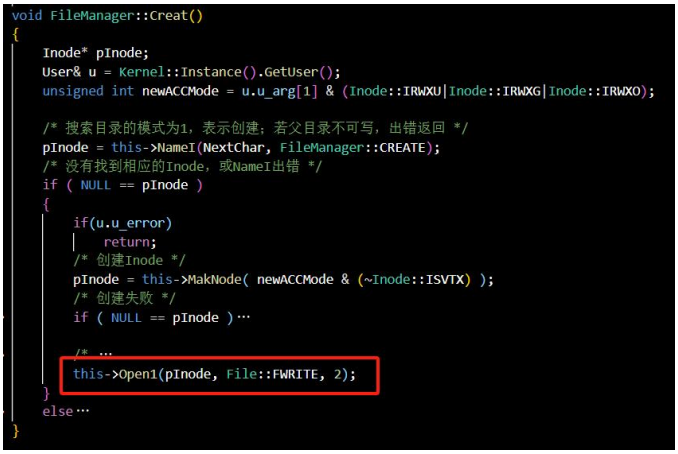
代码 1 执行结果



将写操作完成之后的关闭文件和打开文件两句代码注释掉运行结果

Q：文件创建成功之后，为什么没有直接完成读写操作，而是写过之后，先关闭，再重新打开？将写操作完成之后的关闭文件和打开文件两句代码注释掉，将得到错误输出。尝试解释其中的原因，并从 UNIX V6++的代码中找到依据。

A：在 creat 函数的源代码中，创建一个文件后，默认以“FWRITE”写方式打开，此时并没有读权限。因此需要写过之后关闭，再以可读可写的权限方式打开，才能正确读出。如果注释掉，会因为没有读权限而读出失败。



```
void FileManager::Creat()
{
    Inode* pInode;
    User& u = Kernel::Instance().GetUser();
    unsigned int newACCMode = u.u_arg[1] & (Inode::IRWXU|Inode::IRWXG|Inode::IRWXO);

    /* 搜索目录的模式为1，表示创建；若父目录不可写，出错返回 */
    pInode = this->NameI(NextChar, FileManager::CREATE);
    /* 没有找到相应的Inode，或NameI出错 */
    if ( NULL == pInode )
    {
        if(u.u_error)
            return;
        /* 创建Inode */
        pInode = this->MakNode( newACCMode & (~Inode::ISVTX) );
        /* 创建失败 */
        if ( NULL == pInode )…

        /* …
        this->Open1(pInode, File::FWRITE, 2);
    }
    else…
}
```

（2）完成实验 4.2，并回答其中的 3 个问题。

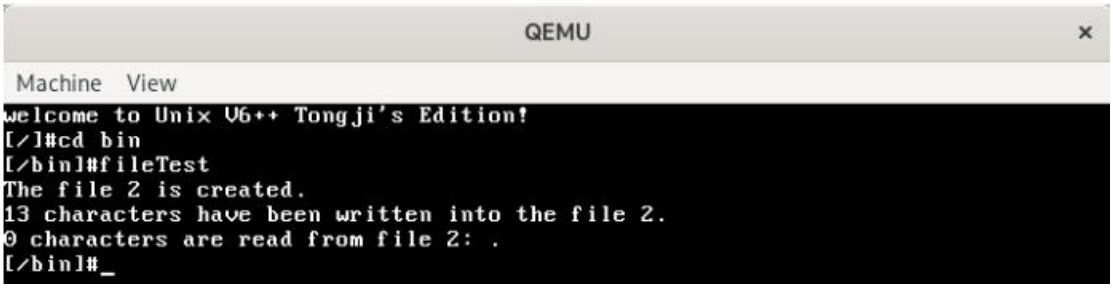
代码 2 执行结果

Q1:说明在代码 2 中，父子进程如何实现对文件的读写权限和读写指针的共享；

A1:在代码 2 中，父子进程共享的是同一个 File 结构和同一个 iNode 节点，所以对文件具有相同的读写权限和读写指针，从而实现共享。

Q2:父进程被唤醒重新上台后，为什么要执行 seek 语句，如果没有这条语句，程序最后的输出是什么样的？为什么？

A2:因为父子进程使用同一个文件读写指针。如果父进程重新上台后不执行 seek 语句，那么文件读写指针仍然停留在子进程完成文件写操作后的文件尾。读操作无法读回“Hello World！”。最后的输出为：



注释掉 seek 语句后的执行结果

Q3:代码 2 中，父子进程执行的 close 操作有何不同？

A3:子进程执行 close 操作时，因为此时内存 iNode 和 File 结构还有父进程在使用，所以只是释放打开文件描述符 fd，递减 File 结构引用计数 f_count，发现未减到 0，则操作结束。

父进程之后执行 close 操作时，释放打开文件描述符 fd，递减 File 结构引用计数 f_count，发现减到 0，则释放该 File 结构，递减 iNode 节点引用计数 i_count，发现减到 0，释放 iNode 节点。

（3）完成实验 4.3，并回答其中的 3 个问题。

代码 3 执行结果

Q1:说明在你的代码中，父子进程对 Jessy 文件的共享方式；

A1:在代码 3 中，父子进程分别先后打开 Jessy 文件，所以有自己的内存 Inode 节点和 File 结构。所以父子进程的文件读写指针不同，各自有各自的，同时权限也不同：父进程为读权限，子进程为写权限。

Q2:在这样的共享方式中，父进程被唤醒重新上台后，是否还需要执行 seek 函数，为什么？

A2:因此这种共享方式，父子进程各自有自己的读写指针，所以子进程的并不会影响父进程的文件指针。所以父进程被唤醒重新上台后，不再需要执行 seek 函数。

Q3:此处父子进程关闭文件的操作有何不同？

A3:在代码 3 中，子进程和父进程执行的 close 操作是一样的，都是释放打开文件描述符 fd，递减 File 结构引用计数 f_count,发现减到 0，释放该 File 结构；递减 iNode 节点发现计数 i_count 减到 0，释放该 iNode 节点。